

7교시: GitHub Actions 2 - Secrets, Docker Hub Push, 유사 도구

Week 3 Day 3 Lesson 7 - 로그 분산 문제

MSA 환경에서는 하나의 사용자 요청이 여러 서비스와 DB를 거치며, 각 서비스의 로그 파일이 분산됩니다. 요청 식별 정보가 없으면 로그를 이어서 원인을 분석하기 어렵습니다.

frontend logs /var/log/frontend/app.log

api logs /var/log/api/app.log

worker logs /var/log/worker/app.log

DB logs /var/log/postgresql/postgresql.log

핵심 아이디어

모든 로그에 동일한 correlation id를 포함하면 여러 로그를 연결하여 요청의 흐름을 추적할 수 있습니다.

문제: request id가 없을 때

- 각 서비스의 로그에 공통 식별자가 없어 연결 불가
- 시간만으로 추정해야 하므로 신뢰도 낮음
- 문제 발생 시 원인 파악 시간이 크게 증가

어느 요청의 로그인지 알 수 없음!

요청 추적 방법

- frontend가 수신 시 correlation id 생성 (UUID)
- 모든 하위 호출에 헤더로 전파 (예: X-Correlation-ID)
- 모든 서비스 로그에 correlation id 포함
- 하나의 id로 전체 요청 흐름 연결 및 분석

없는 경우: 추적 불가 (시간 추정만 가능)

① 시간으로만 유사 구간 추정 (불확실)

정확한 연결이 불가능하여 원인 분석에 많은 시간 소요

있는 경우: correlation id로 요청 추적

① 모든 로그에 동일한 correlation id 포함

정확한 연결로 빠른 원인 분석 및 요청 흐름 파악 가능

로그 포맷 예시 (JSON)

```
{
  "timestamp": "2025-06-09T10:00:00.124Z",
  "level": "INFO",
  "service": "api",
  "correlation_id": "3f1e9c2a-7b5d-4b12-b2a1-9a0c8e6d4f11",
  "request_id": "req-8f3c2a111",
  "message": "call order service",
  "method": "GET",
  "path": "/api/orders",
  "duration_ms": 86
}
```

모범 사례 (Best Practices)

- correlation id는 모든 로그의 기본 필드로 포함
- 로그 포맷은 구조화(JSON)로 남기기
- 시간(UTC), level, service, message 함께 기록
- 불필요한 개인정보/민감정보는 로그에 남기지 않기
- 분산 추적(Tracing) 도구 도입 고려 (추후 학습)

수업 목표

- GitHub Actions에서 Docker Hub에 image를 push하는 흐름을 설명한다.
- Docker Hub secrets, image tag, buildx, push 로그를 확인한다.
- Docker Hub에서 생성된 image를 확인하고 로컬에서 pull/run한다.
- GitHub Secrets의 장점과 단점을 설명한다.
- Jenkins, TeamCity, AWS CodePipeline과 GitHub Actions를 비교한다.

Docker Hub 준비

필요한 것:

항목	설명
Docker Hub 계정	image repository owner
Access Token	GitHub Actions에서 login할 token
GitHub Secrets	token을 안전하게 저장
repository name	w3d3-dockerhub-app
visibility	public/private 선택

수업에서는 Docker Hub password를 workflow나 문서에 쓰지 않고 access token을 사용한다.

GitHub repository secrets:

Secret	값
DOCKERHUB_USERNAME	Docker Hub username
DOCKERHUB_TOKEN	Docker Hub access token

GitHub Secrets 장단점

장점	설명
repo에 token을 저장하지 않음	source code 노출 위험 감소
Actions에서 쉽게 참조	<code>\${{ secrets.NAME }}</code>
log masking 지원	실수로 출력해도 일부 보호
environment별 분리 가능	dev/stage/prod secret 분리

단점	설명
값 재확인 불가	저장 후 UI에서 다시 볼 수 없음
권한 관리 필요	누가 secret을 수정할 수 있는지 통제 필요
rotation 필요	token 만료/교체 절차 필요
만능 보안 아님	workflow가 악성으로 바뀌면 secret 사용 위험

Workflow 핵심 구간

```
- name: Login to Docker Hub
  uses: docker/login-action@v3
  with:
    username: ${ secrets.DOCKERHUB_USERNAME }
    password: ${ secrets.DOCKERHUB_TOKEN }

- name: Build and push image
  uses: docker/build-push-action@v6
  with:
    context: week3/day3/labs/dockerhub-app
    push: true
```

실행 방법

방법 1: Git tag push

```
git tag v0.1.0
git push origin v0.1.0
```

방법 2: GitHub UI에서 manual run

Actions -> w3d3-dockerhub-publish -> Run workflow

Actions에서 확인할 것

위치	Evidence
workflow run	success/failure
Prepare image metadata	image name과 version
Login to Docker Hub	login 성공
Build and push image	pushed digest, tag
Show pull command	pull/run 명령

수동 배포의 불편함

GitHub Actions가 없으면 사람이 다음을 직접 해야 한다.

```
테스트 실행
Docker build
Docker login
Docker tag
Docker push
Docker Hub 확인
서버에서 pull
서버에서 run
검증 curl
```

문제:

문제	설명
누락	test나 scan을 빼먹기 쉽다
불일치	사람마다 명령과 tag 기준이 다르다
증거 부족	누가 어떤 image를 push했는지 흐리다
secret 위험	로컬 shell history나 화면에 token 노출 가능
재현 어려움	같은 절차를 반복하기 어렵다

Actions는 이 절차를 YAML로 고정하고 실행 로그를 남긴다.

Docker Hub에서 확인

Docker Hub UI:

Repositories -> DOCKERHUB_USERNAME/w3d3-dockerhub-app -> Tags

확인:

항목	기준
0.1.0 tag	version tag 생성
latest tag	latest도 생성
pushed time	workflow 실행 시각과 일치

로컬 pull/run 검증

Docker Hub repository가 public이면 바로 pull할 수 있다.

```
docker pull DOCKERHUB_USERNAME/w3d3-dockerhub-app:0.1.0
docker rm -f w3d3-dockerhub-app 2>/dev/null || true
docker run -d --name w3d3-dockerhub-app -p 18088:8080 DOCKERHUB_USERNAME/w3d3-dockerhub-app:0.1.0
curl -s http://localhost:18088/health
docker rm -f w3d3-dockerhub-app
```

주의:

DOCKERHUB_USERNAME은 실제 계정으로 바꾼다.

Docker Hub repository가 private이면 pull하는 환경에서도 인증이 필요하다.

```
docker login -u DOCKERHUB_USERNAME
docker pull DOCKERHUB_USERNAME/w3d3-dockerhub-app:0.1.0
```

```
docker rm -f w3d3-dockerhub-app 2>/dev/null || true
docker run -d --name w3d3-dockerhub-app -p 18088:8080 DOCKERHUB_USERNAME/w3d3-dockerhub-app:0.1.0
curl -s http://localhost:18088/health
docker rm -f w3d3-dockerhub-app
docker logout
```

docker login에서 password 대신 Docker Hub access token을 입력한다. private image는 registry에 push되어 있어도 인증하지 않은 PC, 서버, Kubernetes node에서는 pull할 수 없다.

실패 사례

실패	원인 후보
login failed	secret 이름/토큰 오류
denied requested access	Docker Hub repo 권한 문제
pull access denied	private repo인데 docker login을 하지 않음
build context not found	workflow path 오류
tag가 안 보임	push event/tag 조건 불일치

비슷한 도구

도구	특징	적합한 경우
GitHub Actions	GitHub repo와 통합이 쉬움	GitHub 중심 팀
Jenkins	오래된 표준, 플러그인 풍부	자체 운영/커스터마이징 필요
TeamCity	JetBrains 계열, 엔터프라이즈 CI 기능	상용 CI 운영 조직
AWS CodePipeline	AWS IAM/CodeBuild/ECR/ECS와 통합	AWS 중심 배포

도구보다 중요한 것은 pipeline의 단계다.

```
source -> test -> security scan -> build -> push -> deploy -> verify
```

핵심 포인트

CI가 image를 build했다는 것과 registry에 push되어 pull/run 가능하다는 것은 다르다. 마지막 검증은 항상 registry에서 받은 image를 실행하는 것이다.

Evidence Note

```
# W3D3S7 Docker Hub Push
- workflow run:
- image name:
- tag:
- Docker Hub tag visible:
- repository visibility:
- private pull auth:
- docker pull:
- local run result:
- failure evidence:
```